

## Detailed Implementation Plan

Do not apply every change in one uncontrolled rewrite. First repair the evaluation protocol, then introduce model corrections incrementally. Otherwise, any accuracy improvement will be impossible to attribute scientifically.

### Phase 0 — Freeze and validate the baseline

Before modifying training:

- Record the current Git commit, environment and complete command-line configuration.
- Preserve the 200-epoch log as the baseline.
- Record:
  - Subjects: 956
  - ASD: 455
  - NC: 501
  - ROIs: 90
  - Node features: 3
  - Batch size: 32
  - Current best accuracy: 53.15%
- Run the classical ML baseline again using exactly the same fixed folds later used by A-GCL.
- Assign every subject a stable numerical `subject_id`.
- Sort filenames before dataset construction.
- Regenerate `data/processed/data.pt` after changing the dataset loader.

Files:

- [abideDataset.py](#)
- [agcl\\_ABIDE\\_queue.py](#)

Required data assertions:

X shape = [90, 3]

E shape = [90, 90]

X contains finite values

E contains finite values

X range = [0, 1]

E range = [-1, 1]

E approximately symmetric

Labels = {0, 1}

Every subject ID is unique

## **Phase 1 — Repair the evaluation protocol first**

### **1.1 Precompute fixed stratified folds**

Replace:

```
KFold(..., random_state=None)
```

with:

```
StratifiedKFold(  
    n_splits=5,  
    shuffle=True,  
    random_state=args.seed  
)
```

For each outer training partition, create a fixed stratified validation subset:

```
train_test_split(  
    train_val_indices,  
    test_size=0.2,  
    stratify=labels[train_val_indices],  
    random_state=args.seed + fold_id  
)
```

Save the resulting indices:

```
splits/
```

└─ abide\_5fold\_seed123.npz

Every epoch and every experimental variant must load the same indices. The split file should contain:

fold\_0\_train

fold\_0\_val

fold\_0\_test

...

fold\_4\_train

fold\_4\_val

fold\_4\_test

Add tests verifying:

- No overlap between train, validation and test.
- Every subject appears in one outer test fold.
- Class proportions are approximately preserved.
- Loading the splits repeatedly produces identical hashes.

## 1.2 Correct SVM scaling and parameter search

Currently, StandardScaler sits outside GridSearchCV, causing information from the inner validation folds to affect scaling.

Use:

```
svm_pipeline = make_pipeline(
```

```
    StandardScaler(),
```

```
    LinearSVC(
```

```
        dual=False,
```

```
        fit_intercept=True,
```

```
        max_iter=10000
```

```
    )
```

```
)
```

```

classifier = GridSearchCV(
    svm_pipeline,
    {
        "linearsvc__C": [0.1, 0.5, 1, 5, 10]
    },
    cv=StratifiedKFold(
        n_splits=5,
        shuffle=True,
        random_state=seed
    ),
    scoring="accuracy"
)

```

The listed (C) values reproduce the paper's stated SVM search space. A wider grid can be reported as a separate ablation.

### 1.3 Honest checkpoint selection

At every evaluation interval:

1. Extract embeddings.
2. Train SVMs only on training indices.
3. Calculate validation metrics.
4. Do not use test accuracy for checkpoint selection.
5. Save the model when mean validation accuracy improves.

if val\_accuracy > best\_val\_accuracy:

```
best_val_accuracy = val_accuracy
```

```
best_epoch = epoch
```

```
save_checkpoint(...)
```

After training:

1. Reload the checkpoint selected by validation accuracy.
2. Evaluate the test folds once.
3. Calculate all metrics from this one checkpoint.

Report:

Accuracy

AUC

F1-score

Sensitivity

Specificity

Do not select separate epochs for different metrics.

Store the real training epoch:

`best_epoch = epoch`

Do not report the index of the metrics array as the epoch.

Primary file:

- [embedding\\_evaluation.py](#)

---

## Phase 2 — Correct augmented-view construction

Create one reusable augmentation function so the view-training and model-training stages cannot implement augmentation differently.

Suggested new file:

`unsupervised/augmentation.py`

Function:

```
def construct_augmented_edge_weights(  
    original_edge_weight,  
    edge_mask
```

);

```
assert original_edge_weight.shape == edge_mask.shape
```

```
augmented_edge_weight = original_edge_weight * edge_mask
```

```
return augmented_edge_weight
```

Replace both augmented forward passes with:

```
aug_edge_weight = batch.edge_weight * edge_mask
```

```
z_aug, _ = model(  
    batch.batch,  
    batch.x,  
    batch.edge_index,  
    None,  
    aug_edge_weight  
)
```

Do this in:

- View-learner update.
- Main-model update.
- Non-memory implementation, if it will still be maintained.

Files:

- [agcl\\_ABIDE\\_queue.py](#)
- [agcl\\_ABIDE.py](#)

### Unit tests

B = all ones → E<sub>aug</sub> exactly equals E

B = all zeros → E<sub>aug</sub> exactly equals zero

$B \in (0,1) \rightarrow |E_{aug}| \leq |E|$

Negative E  $\rightarrow$  sign remains negative

Positive E  $\rightarrow$  sign remains positive

Use:

```
edge_mask = edge_mask.squeeze(-1)
```

Do not use unrestricted `.squeeze()`, because that could remove unintended dimensions for small test cases.

---

## Phase 3 — Correct the regularizer and monitor mask collapse

### 3.1 Separate deterministic probability from sampled mask

The view learner produces edge logits ( $l$ ). Define:

```
[  
  \mu = \operatorname{sigmoid}(l)  
]
```

Use ( $\mu$ ) for regularization:

```
keep_probability = torch.sigmoid(symmetric_edge_logits)
```

```
keep_regularizer = keep_probability.mean()
```

Use the noisy Logistic-Concrete sample only for constructing the augmented graph:

```
edge_mask = sigmoid(  
    (symmetric_edge_logits + logistic_noise) / temperature  
)
```

Do not calculate the regularizer from the sampled ( $B$ ), because that adds unnecessary sampling noise and does not match the paper's ( $R(\mu)$ ).

### 3.2 Implement explicit regularizer modes

Because the paper and released code disagree, do not silently overwrite the behaviour. Add a configuration option:

```
--regularizer_mode paper_keep
```

```
--regularizer_mode adgcl_budget
```

```
--regularizer_mode target_keep
```

### **Paper-aligned mode**

The view learner maximizes:

$$\begin{aligned} & [ \\ & J_{\text{view}} \\ & L_{\text{batch}} \\ & + \\ & \lambda_2 L_{\text{memory}} \\ & \lambda_1 \operatorname{mean}(\mu) \\ & ] \end{aligned}$$

Implementation:

```
view_score = (
    batch_contrastive_loss
    + args.cr_lambda * memory_loss
    - args.reg_lambda * keep_regularizer
)
```

```
view_optimization_loss = -view_score
```

### **Existing AD-GCL mode**

Preserve it only for comparison:

```
drop_regularizer = (1 - edge_mask).mean()
```

```
view_score = (
    batch_contrastive_loss
    + args.cr_lambda * memory_loss
    - args.reg_lambda * drop_regularizer
)
```



)

### Target-sparsity mode

Use only if both paper and released-code modes collapse:

```
[  
  R_{\text{target}}=  
  (\operatorname{mean}(\mu)-\rho)^2  
]
```

```
target_penalty = (  
    keep_probability.mean() - args.target_keep_ratio  
).pow(2)
```

This is an engineering improvement, not a reproduction of the published objective, so it must be labelled accordingly.

### 3.3 Add mask diagnostics

Log every epoch:

Expected keep probability: mean/std

Sampled keep fraction

Sampled drop fraction

Minimum and maximum  $\mu$

$\mu$  quantiles: 5%, 25%, 50%, 75%, 95%

Mask entropy

Percentage  $\mu < 0.1$

Percentage  $\mu > 0.9$

Generate an alert when:

keep\_mean > 0.95 for 10 epochs

or

keep\_mean < 0.05 for 10 epochs

That detects all-keep and all-drop collapse.

---

## Phase 4 — Rebuild the memory queue

### 4.1 Queue structure

Replace the current queue with:

class MemoryBank:

```
def __init__(self, max_length, feature_dim, device):
    self.features = torch.zeros(
        max_length,
        feature_dim,
        device=device
    )
    self.subject_ids = torch.full(
        (max_length,),
        -1,
        dtype=torch.long,
        device=device
    )
    self.valid = torch.zeros(
        max_length,
        dtype=torch.bool,
        device=device
    )
    self.pointer = 0
```

The queue must not use:

requires\_grad=True

and should not update through:

```
tensor.data
```

Use:

```
@torch.no_grad()
```

```
def enqueue(self, features, subject_ids):
```

```
    features = features.detach()
```

#### **4.2 Correct queue order**

Correct batch sequence:

1. Calculate view-learner loss using previous queue
2. Update view learner
3. Calculate main-model loss using the same previous queue
4. Update main model
5. Enqueue the current detached augmented embeddings
6. Continue to the next batch

Therefore:

```
memory_loss = calculate_memory_loss(
```

```
    z_original,
```

```
    z_augmented,
```

```
    memory_bank,
```

```
    subject_ids
```

```
)
```

```
# Optimizer updates happen here
```

```
memory_bank.enqueue(
```

```
    z_augmented.detach(),
```

```
    subject_ids
```

)

The current batch must not enter memory before its loss is calculated.

#### **4.3 Exclude the same subject**

For each anchor:

```
negative_mask = (  
    memory_bank.valid  
    & (memory_bank.subject_ids != anchor_subject_id)  
)
```

This prevents an old embedding of the same subject from being treated as a negative.

#### **4.4 Stable memory loss**

Replace:

```
torch.log(torch.exp(logits).sum(...))
```

with:

```
torch.logsumexp(logits, dim=1)
```

This is numerically safer.

When no valid memory negatives exist:

```
memory_loss = z_original.new_zeros(()))
```

Optionally provide a strict released-code mode that retains zero-filled cold-start entries, but use valid entries as the corrected implementation.

#### **Queue tests**

Verify:

- Queue contains only detached tensors.
- Queue has no gradients.
- Pointer wraps correctly.
- Valid count increases correctly.
- The current batch is absent during its loss.

- Matching subject IDs are excluded.
- Queue size never exceeds 256.
- The first batch runs without memory negatives.
- After eight 32-subject batches, the queue is full.

---

## Phase 5 — Enforce a symmetric FC mask

The robust implementation should not rely on edge ordering.

For every directed edge ( $i \rightarrow j$ ), form an unordered pair identifier:

```
[
p_{ij}=(\min(i,j),\max(i,j))
]
```

Then:

1. Group both directions using the pair identifier.
2. Average their logits:

```
[
l^{sym}_{ij}
\frac{l_{ij}+l_{ji}}{2}
]
```

3. Generate one Logistic-Concrete noise value per unordered pair.
4. Produce one mask value.
5. Broadcast it to both directions.

Required result:

```
[
B_{ij}=B_{ji}
]
```

and therefore:

```
[
E^{aug}_{ij}=E^{aug}_{ji}
]
```

Keep diagonal handling explicit. For strict paper alignment, include diagonal entries in the mask and regularizer because the paper uses  $(M^2)$  entries. Log off-diagonal statistics separately.

Tests:

```
B[i,j] == B[j,i]
```

```
Eaug[i,j] == Eaug[j,i]
```

Gradients reach both directional logits

Self-loop handling is deterministic

Different subjects do not share edge pairs

Primary file:

- [view learner.py](#)

## Phase 6 — Evaluate the correct representation

Add embedding extraction to the complete GInfoMinMax model.

### Encoder representation

```
[
h=\operatorname{Pool}(\operatorname{GIN}(G))
]
```

### Projected representation

```
[
z=\operatorname{ProjectionHead}(h)
]
```

Implementation:

```
def get_embeddings(self, loader, device, representation="z"):
    self.eval()
```

```
embeddings = []
```

```
labels = []
```

```
with torch.no_grad():
```

```
    for batch in loader:
```

```
        h, _ = self.encoder(
            batch.batch,
            batch.x,
            batch.edge_index,
            None,
            batch.edge_weight
        )
```

```
        if representation == "h":
```

```
            output = h
```

```
        elif representation == "z":
```

```
            output = self.proj_head(h)
```

```
        else:
```

```
            raise ValueError(representation)
```

```
        embeddings.append(output.cpu())
```

```
        labels.append(batch.y.cpu())
```

```
    return torch.cat(embeddings), torch.cat(labels)
```

At every fixed checkpoint, evaluate both:

SVM(h)

SVM(z)

Use (z) as the primary paper-aligned result. Report (h) as an ablation.

Change:

```
ee.kf_embedding_evaluation(model.encoder, dataset)
```

to an interface that receives the complete model:

```
ee.kf_embedding_evaluation(  
    model,  
    dataset,  
    representation="z",  
    splits=fixed_splits  
)
```

Files:

- [ginfominmax.py](#)
- [embedding\\_evaluation.py](#)

---

## Phase 7 — Make encoder deviations configurable

Do not immediately delete every extra encoder operation. Convert them into explicit options:

```
--normalize_nodes true/false
```

```
--message_relu true/false
```

```
--post_bn_relu true/false
```

```
--drop_ratio 0.0/0.3
```

### Current-code configuration

```
normalize_nodes = True
```

```
message_relu = True
```

```
post_bn_relu = True
```

```
drop_ratio = 0.3
```



### Paper-aligned configuration

normalize\_nodes = False

message\_relu = False

post\_bn\_relu = False

drop\_ratio = 0.0

Apply the same encoder configuration to:

- Main-model WGIN encoder.
- View-learner WGIN encoder.

Files:

- [tu\\_encoder.py](#)
- [wgin\\_conv.py](#)

Recommended ablations:

#### Variant Change

- |    |                                 |
|----|---------------------------------|
| A0 | Current encoder                 |
| A1 | Remove node L2 normalization    |
| A2 | Remove message-level ReLU       |
| A3 | Remove post-BN ReLU             |
| A4 | Set dropout to zero             |
| A5 | Apply all paper-aligned changes |

Because message ReLU and post-BN ReLU interact, also run the combined A2+A3 condition.

---

### Phase 8 — Refactor the training loop

Corrected batch pseudocode:

for batch in dataloader:

    batch = batch.to(device)

```
subject_ids = batch.subject_id
```

```
# -----
```

```
# 1. Update view learner
```

```
# -----
```

```
freeze(model)
```

```
unfreeze(view_learner)
```

```
model.eval()
```

```
view_learner.train()
```

```
edge_logits = view_learner(...)
```

```
symmetric_logits = symmetrize(edge_logits, batch.edge_index)
```

```
keep_probability = torch.sigmoid(symmetric_logits)
```

```
edge_mask = sample_logistic_concrete(symmetric_logits)
```

```
augmented_edge_weight = batch.edge_weight * edge_mask
```

```
z, _ = model(..., batch.edge_weight)
```

```
z_aug, _ = model(..., augmented_edge_weight)
```

```
batch_loss = model.calc_loss(z, z_aug)
```

```
memory_loss = calculate_memory_loss(
```

```
    z,
```

```
    z_aug,
```

```
memory_bank,  
subject_ids  
)
```

```
view_score = (  
    batch_loss  
    + cr_lambda * memory_loss  
    - reg_lambda * keep_probability.mean()  
)
```

```
view_optimizer.zero_grad(set_to_none=True)  
(-view_score).backward()  
view_optimizer.step()
```

```
# -----
```

```
# 2. Update main model
```

```
# -----
```

```
unfreeze(model)
```

```
freeze(view_learner)
```

```
model.train()
```

```
view_learner.eval()
```

```
with torch.no_grad():
```

```
    edge_logits = view_learner(...)
```

```
    symmetric_logits = symmetrize(...)
```

```

edge_mask = sample_logistic_concrete(
    symmetric_logits
)

augmented_edge_weight = (
    batch.edge_weight * edge_mask
)

z, _ = model(..., batch.edge_weight)
z_aug, _ = model(..., augmented_edge_weight)

batch_loss = model.calc_loss(z, z_aug)
memory_loss = calculate_memory_loss(
    z,
    z_aug,
    memory_bank,
    subject_ids
)

model_loss = (
    batch_loss
    + cr_lambda * memory_loss
)

model_optimizer.zero_grad(set_to_none=True)
model_loss.backward()

```

```
model_optimizer.step()
```

```
# -----
```

```
# 3. Update queue last
```

```
# -----
```

```
memory_bank.enqueue(
```

```
    z_aug.detach(),
```

```
    subject_ids
```

```
)
```

`model.eval()` and `view_learner.eval()` alone do not freeze parameters. Explicit freezing avoids unnecessary gradient calculation.

---

## Phase 9 — Controlled experiment sequence

Use identical splits, seeds, epochs and SVM settings for every variant.

### Core correction experiments

#### Experiment Added correction

C0 Existing training + repaired evaluation only

C1 C0 + ( $E \odot B$ )

C2 C1 + paper-aligned regularizer

C3 C2 + repaired memory queue

C4 C3 + symmetric mask

C5 C4 + projected ( $z$ ) as primary embedding

Evaluate both ( $h$ ) and ( $z$ ) at every stage, but designate ( $z$ ) as primary.

### Encoder experiments

Run A0–A5 on top of C5.

## **Suggested compute protocol**

Screening:

3 seeds

100 epochs

Evaluation every 5 epochs

Validation-selected checkpoint

Final configuration:

5 seeds

Full planned training duration

Same fixed five folds

Test evaluated only after configuration is frozen

Suggested seeds:

123, 456, 789, 2026, 3407

Do not select the final configuration using test accuracy.

---

## **Phase 10 — Required diagnostics**

Training logs should include:

Batch contrastive loss

Memory loss

Total model loss

View objective

Keep-probability mean/std

Sampled drop rate

Mask entropy

Original–augmented positive cosine similarity

Mean negative cosine similarity

Number of valid queue negatives

Number of same-subject negatives removed

Gradient norm of main model

Gradient norm of view learner

Representation-collapse checks:

Feature variance per dimension

Mean pairwise cosine similarity

Effective embedding rank

Number of near-zero dimensions

$\|z_{\text{original}} - z_{\text{augmented}}\|$

These diagnostics will distinguish:

- Underfitting.
- Representation collapse.
- Mask collapse.
- Excessively easy positive pairs.
- Excessively hard augmentation.
- Broken memory negatives.

---

## Phase 11 — Acceptance criteria

### Code correctness

The implementation is complete when:

- Augmented weights always equal  $(E \odot B)$ .
- Signed FC information is preserved.
- The mask is symmetric.
- The queue contains only previous-batch detached embeddings.
- Same-subject memory entries are excluded.

- No queue tensor requires gradients.
- Fold indices are fixed and stratified.
- Test results do not influence checkpoint selection.
- All metrics come from one checkpoint.
- (z) is evaluated as the paper's primary representation.
- All unit and smoke tests pass.

### **Model behaviour**

Treat the following as warnings:

Keep probability > 0.95

Keep probability < 0.05

Near-zero embedding variance

Mean pairwise cosine approaching 1

Sensitivity collapsing while specificity remains high

Accuracy is not a software-correctness test. A correctly implemented model can still perform poorly.

---

### **Phase 12 — Reproduction claim**

Even after all corrections, the current experiment cannot honestly be described as an exact reproduction of 80.65%, because:

Paper: 987 subjects, 116 AAL1 ROIs

Current: 956 subjects, 90 ROIs

Use one of these descriptions:

- "Implementation-level reproduction on a modified ABIDE-I cohort."
- "A-GCL reproduction using a 90-ROI cerebrum-only AAL configuration."

To claim exact reproduction, obtain the same 116-ROI graph construction and approximately the same 987-subject quality-controlled cohort.



Also, the fact that classical ML achieves approximately 70% does not guarantee that this unsupervised GNN must reach 80%. It proves that diagnostic signal exists in the data; it does not prove that A-GCL will preserve that signal. The controlled experiments above must establish that.